

GPU

Part 2 - programming session

Workshop

April 2024

Table of Content

Review

Quick review of GPU architecture

1

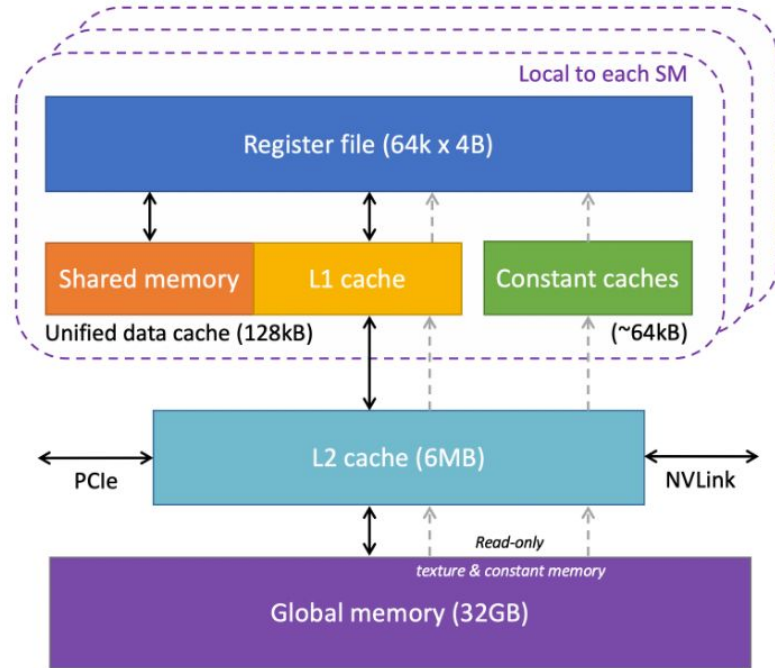
Cuda Kernel Call

2

Cuda memory commands

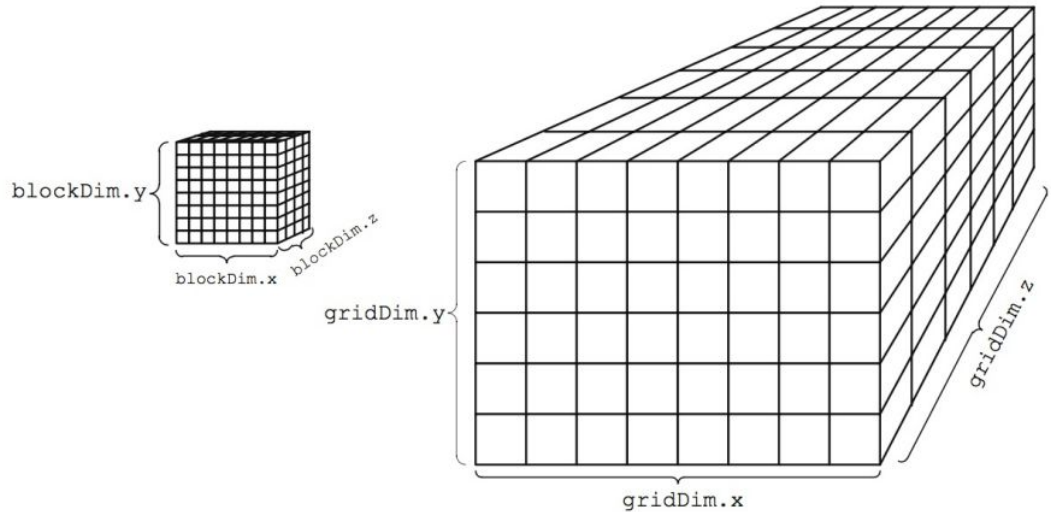
3

Review: Memory Model



Review: Programmer View

- In CUDA, a **kernel** is a function designed to **run in parallel** on a GPU.
- Calling a Kernel requires 2 extra inputs:
 - 1) Block Dim
 - 2) Grid Dim



How does GPU application run?

For sake of simplicity let say we are going to add 2 integers using GPU.

- How to define our function?

s1,2

Kernel

- + **Always** declare as a **void** function.

=> unlike normal function if there is a value return, this value must be wrapped into an array similar to grid and block dimensions.

```
int gpu_add(int a, int b){  
    return a + b; // does it work?  
}
```



Kernel

How CPU should know the existence of a CUDA function?

Kernel

How CPU should know the existence of a CUDA function?

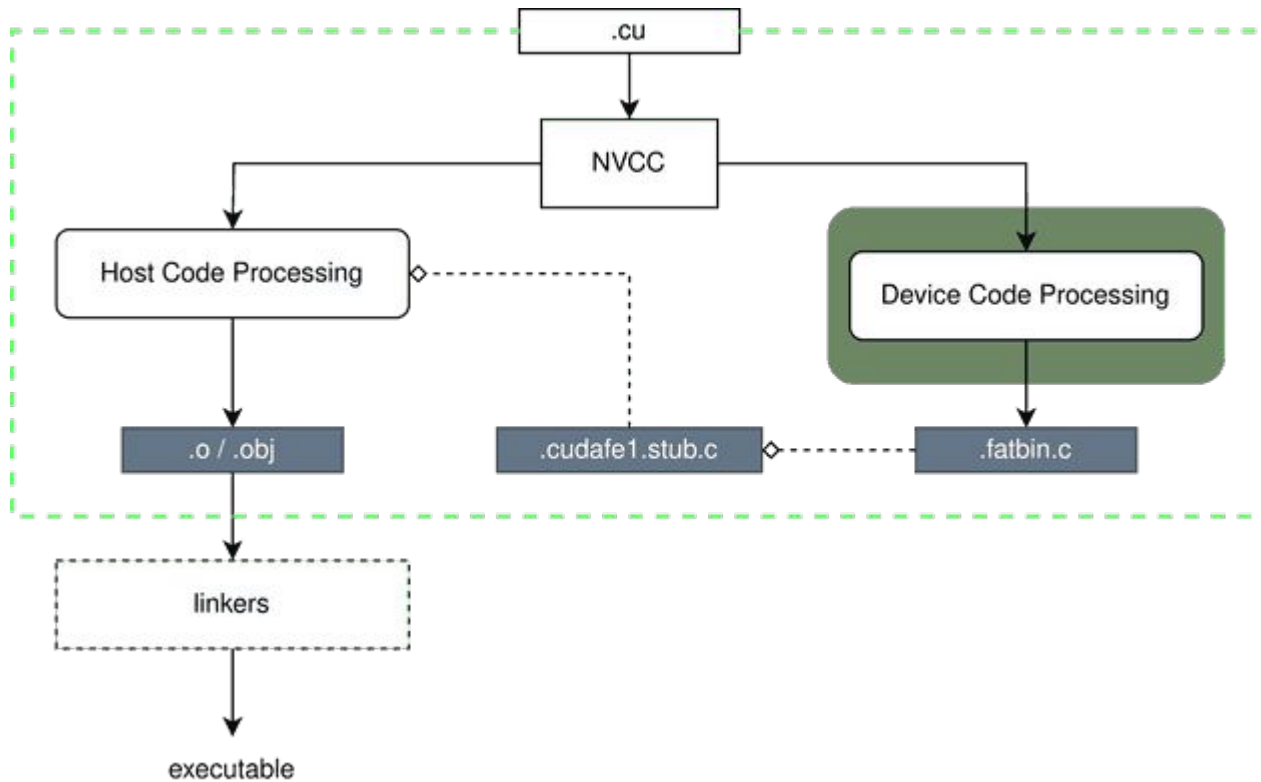
=> by telling the compiler =)

Language Extensions

- `__global__` : both host (cpu) and device(gpu) know this function exist.
 - CPU can call this function.
 - If called, GPU execute it
- `__host__` : only host can see this function.
 - Only visible to host for calling and executing
- `__device__` : only device can see this function.
 - Only visible to device for calling and executing.

Check [HERE](#) (nvidia guide)

nvcc



Kernel, Callable from CPU and executes on GPU

```
__global__  
void gpu_add(int a, int b, int* c){  
    *c = a + b;  
}
```

Why not working?

How GPU knows **a, b and c** variables?

More important question:
why not showing any error?

```
int main(){  
    int c, a = 1, b = 2;  
    gpu_add<<<1,1>>>(a, b, &c);  
}
```

Cuda Error Handling

6.4. Error Handling

This section describes the error handling functions of the CUDA runtime application programming interface.

Functions

`__host__ __device__ const char* cudaGetErrorName ( cudaError\_t error )`
Returns the string representation of an error code enum name.

`__host__ __device__ const char* cudaGetErrorString ( cudaError\_t error )`
Returns the description string for an error code.

`__host__ __device__ cudaError\_t cudaGetLastError ( void )`
Returns the last error from a runtime call.

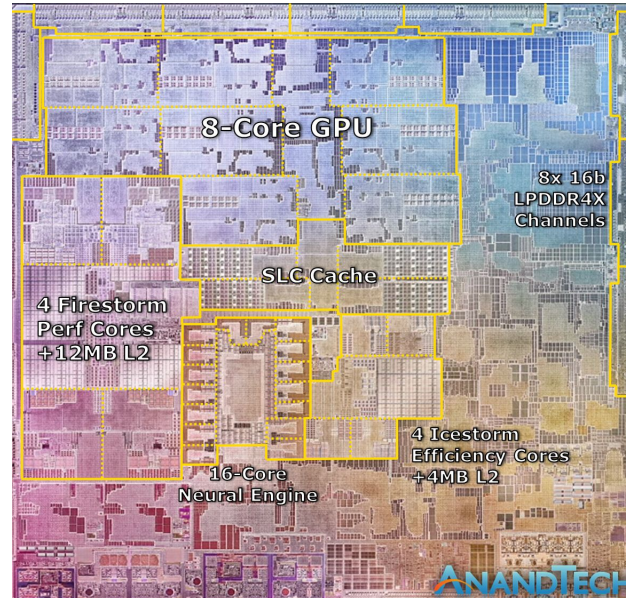
`__host__ __device__ cudaError\_t cudaPeekAtLastError ( void )`
Returns the last error from a runtime call.

Why not working?

How GPU knows **a, b and c variables?**

```
int main(){  
    int c, a = 1, b = 2;  
    gpu_add<<<1,1>>>(a, b, &c);  
}
```

- A) Share memory between CPU(host) and GPU(device)



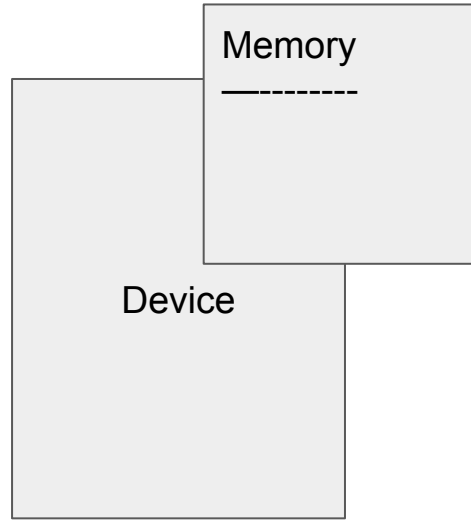
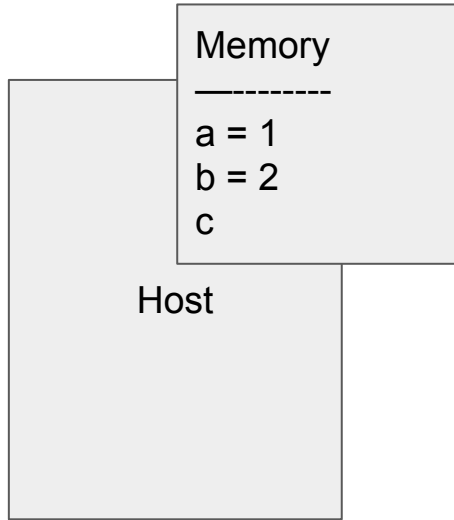
Why not working?

How GPU knows **a, b and c** variables?

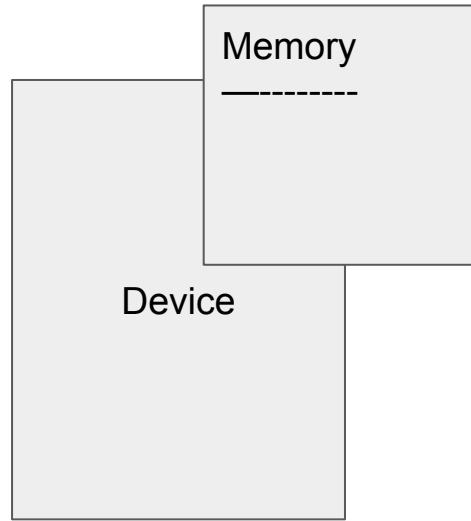
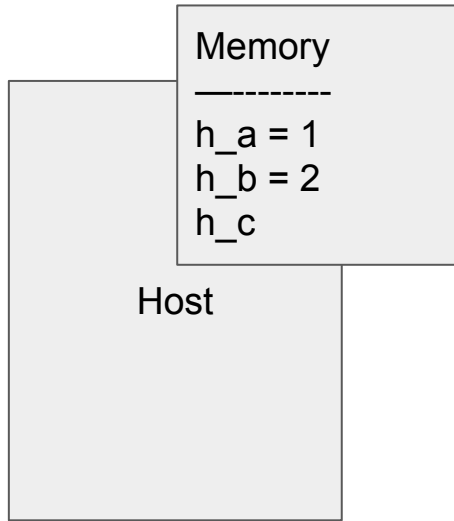
B) By telling the compiler =)



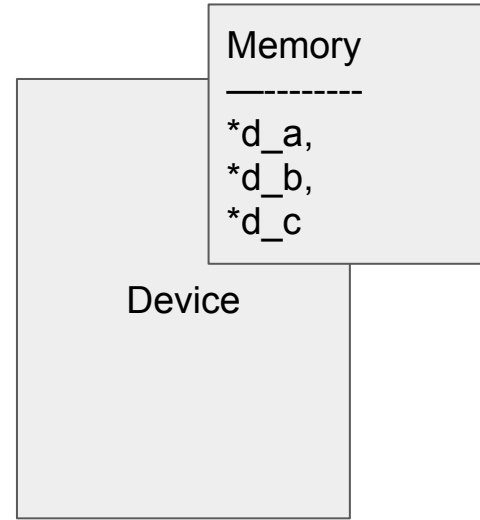
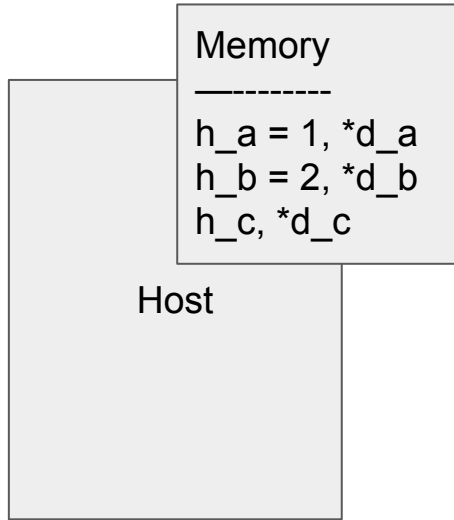
CudaMalloc



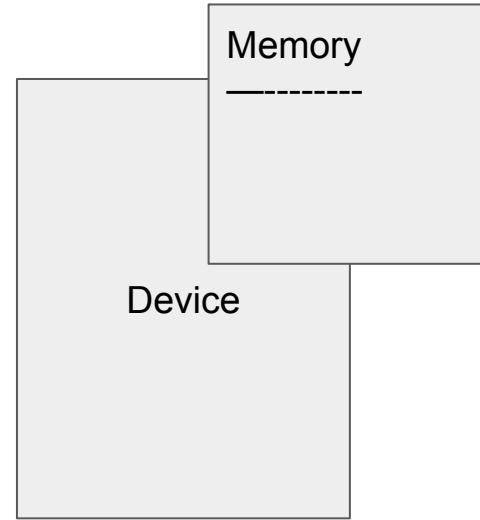
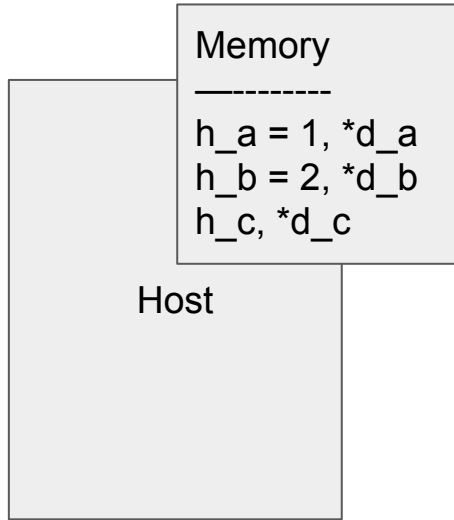
CudaMalloc



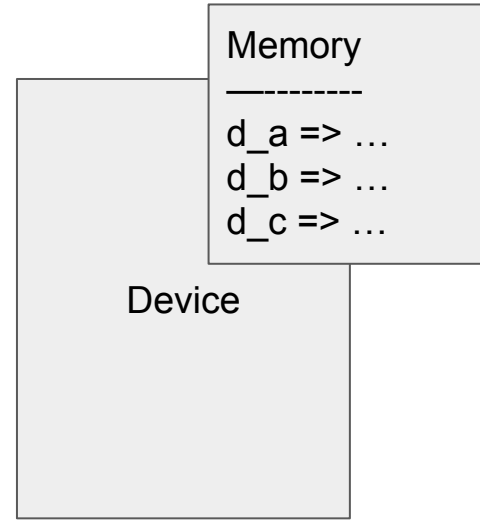
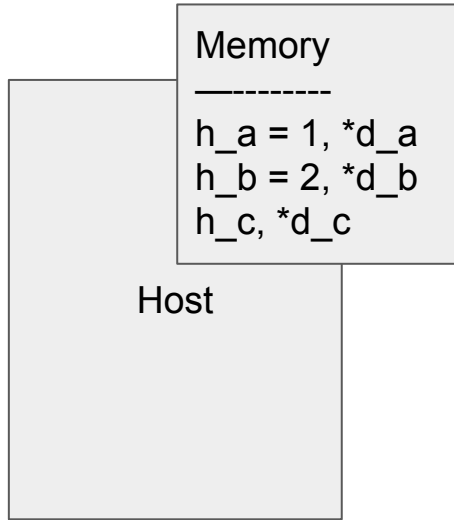
CudaMalloc



CudaMalloc



CudaMalloc



CudaMalloc

```
__host__ __device__ cudaError_t cudaMalloc ( void** devPtr , size_t size )
```

Allocate memory on the device.

Parameters

`devPtr`

- Pointer to allocated device memory

`size`

- Requested allocation size in bytes

GPU knows 3 variables but what about their values?

- We need to copy the value too!

Cuda Malloc

```
__host__ cudaError\_t cudaMemcpy ( void* dst , const void* src , size_t count , cudaMemcpyKind kind )
```

Copies data between host and device.

Parameters

`dst`

- Destination memory address

`src`

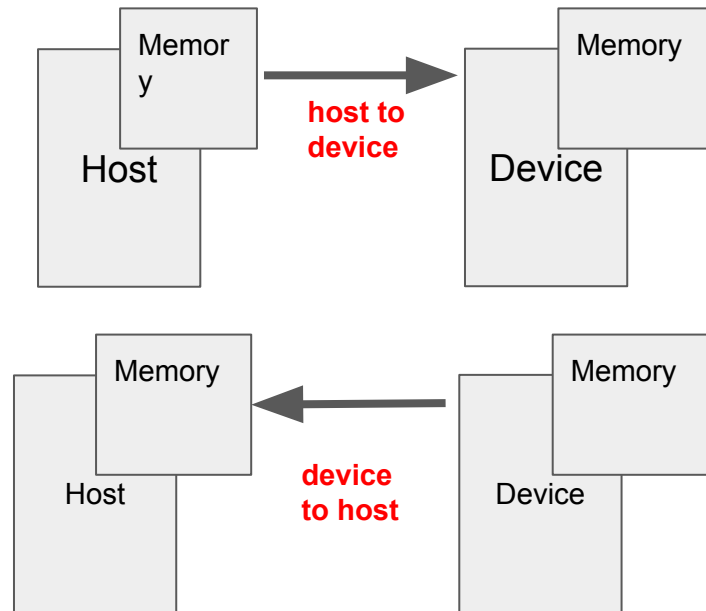
- Source memory address

`count`

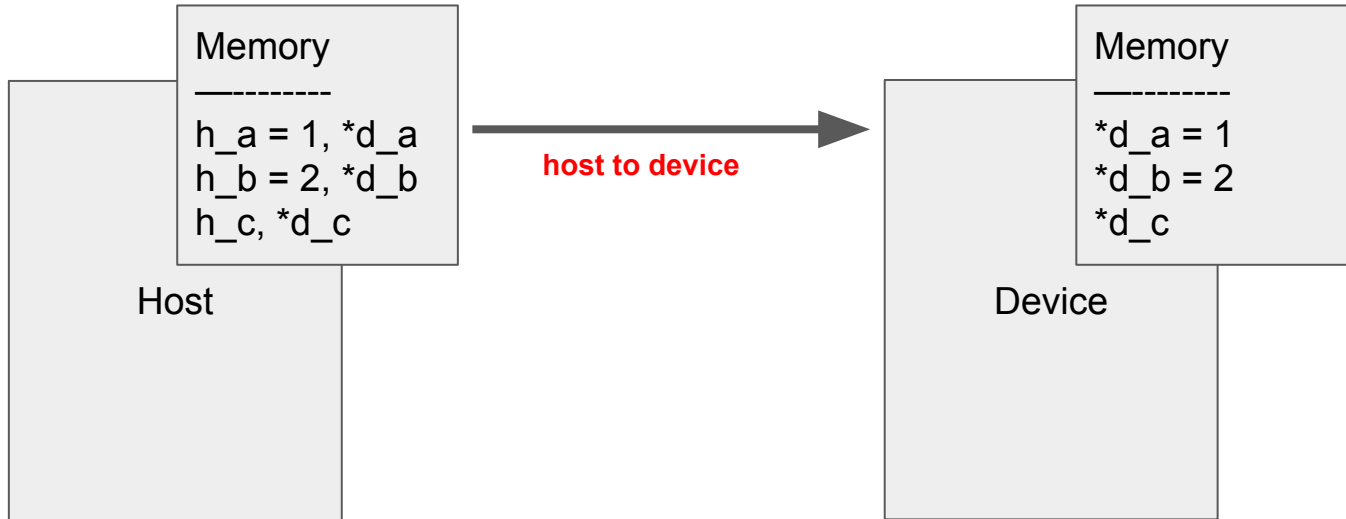
- Size in bytes to copy

`kind`

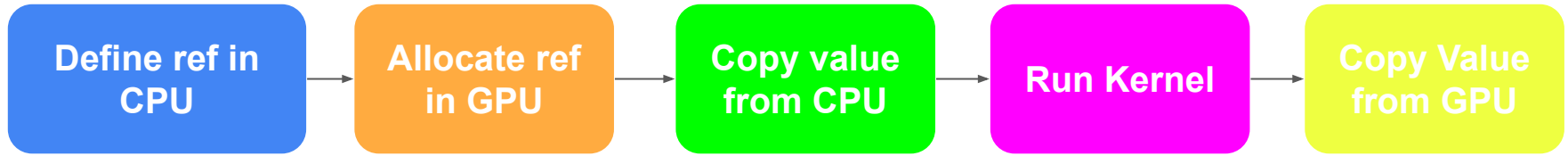
- Type of transfer



CudaMemcpy



Wrap Up



What We Are Going to Do on next Session

- More complex examples